Cassandra vs. MongoDB: A Systematic Review of Two NoSQL Data Stores in Their Industry Uses

Haijing Tu
Department of Communication
Indiana State University
Terre Haute, Indiana
U.S.A.
haijing.tu@indstate.edu

2024 IEEE 7th International Conference on Big Data and Artificial Intelligence (BDAI) | 979-8-3503-5200-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/BDAI62182.2024.10692676

***Abstract*—This paper analyzes two NoSQL data stores: the column-family data store with Cassandra as an example and the document-oriented data store using MongoDB as another example. By conducting a systematic review, this article examines the challenges and opportunities for the most recent industrial applications of these two data stores. As NoSQL data stores, both Cassandra and MongoDB are capable of processing live streaming data. In this systematic review, their applications in storing weather data, traffic data, radio spectrum data, live news data, and spatial data are discussed and compared.**

***Keywords*—DBMS, NoSQL, column-family, document-oriented, Cassandra, MongoDB, industry uses**

I. Introduction

The rise of big data in the past two decades have pushed old data management principles to obsolescence. Big data arrived with 3 Vs: volume, veracity, and variety [1], and the 3 Vs were later expanded to 5 Vs by adding velocity and value [2]. Lately, a few more Vs are added to describe big data, such as visualization, variability, and volatility, etc. [3]. Today's big data infrastructure features distributed and paralleled memory, storage, and processing [4], and challenges old data analytics that relies on shared memory multiprocessing (SMP) [5].

Unlike relational database management systems (DBMS) that run on single node consisting of a collection of cores sharing common memory and disk, non-relational DBMSs run on an architecture configuration to achieve parallelism and load balancing. This is called a "shared-nothing" system with unlimited scalability, which provides great advantage over shared-disk DBMS [5]. In this "shared nothing" architecture, the nodes do not share either memory or disk, they are in a collection of self-contained nodes that are connected to one another through networking. According to Chickerur et al. [6], NoSQL databases are non-relational, schema-free, and possess attributes such as no join, easy replication support, simple API, and eventual consistency.

Stonebraker's research on SQL and NoSQL DBMSs show that their major differences exist in their consistency guarantees, per-server performance, scalability for read vs write loads, automatic recovery from failure of a server, programming convenience, and administrative simplicity [5]. Based on these differences, NoSQL DBMSs have become the natural choice for processing big data [5,7]. At the same time, unlike SQL DBMSs, NoSQL DBMSs do not require ACID properties (Atomicity, Consistency, Isolation, and Durability) [8]. This research will review the most recent industrial applications of two NoSQL databases: MongoDB and Cassandra to compare their strengths and weaknesses, and examine their future roles in Big Data Analytics (BDA).

II. Research Background

At the core of BDA is the drive to uncover meaningful patterns from massive input data for decision making, prediction, and other inferencing [9]. But with the sophisticated nature of big data and its 5 Vs, all choices in DBMSs involves trade-offs per the CAP theorem: consistency, availability and partition tolerance [5,10]. It is necessary to clarify concepts related to data stores and data models before analyzing specific opportunities and challenges related to Cassandra and MongoDB and their uses in the industry.

A. Data Model and Data Store

According to [11], data model is a set of data structures that mainly describes data types, properties, and relationships. The NoSQL data model is classified into four main groups: key-value, column store, document store, and graph database [11–13]. The first three data stores are aggregated-oriented, whereas the last one is not. Data stores are highly distributed, and they are widely used for applications. Of the various NoSQL data stores available, Cassandra and MongoDB are identified with the data processing nature for streaming data [2].

B. Cassandra: A Column Store

Originally developed by Facebook in 2008 using Java, Cassandra is a column-family database. It's query language is CQL (Contextual Query Language), which is SQL based in query nature. According to Sharma et al. [2], Cassandra excels in low latency, high throughput, and no single point failure made possible by peer-peer clustering (the same role for each node).

Based on its flexible data model, no single point of failure, linear scalability, and Hadoop integration, Cassandra is widely used in the video streaming service. According to Carpenter [14], Cassandra was a natural choice for Netflix because it solved the limitations on scalability and the potential for failures of an Oracle database in a single data center. In addition to Netflix,

other prominent Cassandra users included Apple, Facebook, IBM, Instagram, Rackspace, Twitter, Uber, Spotify, Cisco, and eBay etc. [15].

C. MongoDB: A Document Store

Developed by 10gen in 2007 using C++, MongoDB is a document-oriented, open source, NoSQL data store [2]. MongoDB documents use the binary format of JSON (BSON). Unlike Cassandra, it supports advanced and wide variety of indexing, including spatial indexing. It uses MongoDB ad-hoc query language that returns specific fields, and this query language is not SQL based in query nature. According to [2], the MongoDB query "set compass searches by fields, range queries, regular expression search, etc. and may include user-defined complex JavaScript functions" (p. 147).

As a document-oriented database, MongoDB's structure is based on grouping documents into collections. Although documents in a collection can vary in data types, MongoDB is more transparent than column-families where multiple column families can be stored in one row. According to Sharma et al. [2], MongoDB has been widely used by big companies such as MetLife, Craigslist, Forbes, *The New York Times*, Sourceforge, and eBay for storing back-end pages and developing search engine application.

D. Research Goals

Both Cassandra and MongoDB are good fit for storing large data across nodes. Yet they both face challenges in industrial application. According to Shultz [16], the ideal Cassandra application has the following characteristics: Writes exceed reads by a large margin, data are rarely updated and when updates are made they are idempotent, read access is by a known primary key, data can be partitioned via a key that allows the database to be spread evenly across multiple nodes, and there is no need for joins or aggregates. However, Cassandra does not have a PHP driver that can be used to program a system [2].

On the other hand, MongoDB is ideal for real time data integration, big data systems, MapReduce applications, news site forums, and social networking applications [17]. It is also equipped with suitable drivers for most of programming languages used for apps that use MongoDB as their back-end player [2]. However, join is not supported in MongoDB, and there is a limit to nesting: MongoDB users can't perform more than 100 levels of nesting of documents as explained at (Hevodata.

Since databases are at the heart of every application, choosing the ideal data store for data management and processing is a highly critical decision [18]. This paper aims to provide a systematic review of the most recent literature to compare the newest opportunities and limitations brought by Cassandra and MongoDB as NoSQL data stores to data management in the industry. As a systematic review, this paper aims to generate an innovative theoretical comparison by integrating previous studies investigation the most up-to-date industrial applications of these two data stores.

III. Methodology

A. Data Selection

The articles for analysis are collected from iEEE Explore, a database that provides access to content from IEEE (Institute of Electrical and Electronics Engineers) and IET (Institution of Engineering and Technology). Searching "Cassandra and MongoDB" returns 70 results from 2011-2023 in iEEE Xplore. The screening of these articles is based on their relevance to industry uses and recency.

After a thorough review these articles, this study excluded articles published before 2015 and kept 17 articles that discuss industry uses of the two data stores (Table 1).

Table 1: Selected Search Results on Cassandra and MongoDB from iEEE Explore

Article Title	Year	Reference
Performance Analysis of Scaling NoSQL vs SQL: A Comparative Study of MongoDB, Cassandra, and PostgreSQL	2023	[13]
SQL Interface Development for Spatial Data Retrieval on Cassandra	2022	[19]
Comparison of SQL and NoSQL databases with different workloads: MongoDB vs MySQL evaluation	2022	[18]
Comparative Study of MongoDB vs Cassandra in big data analytics	2021	[15]
A Systematic Review of Data Models for the Big Data Problem	2021	[11]
Scalable Real-Time Analytics for IoT Applications	2021	[20]
A survey on RDBMS and NoSQL Databases MySQL vs MongoDB	2020	[8]
Comparison of MongoDB and Cassandra Databases for Spectrum Monitoring As-a-Service	2020	[21]
Big Data Analytics for Processing Real-time Unstructured Data from CCTV in Traffic Management	2020	[22]
Wrapping a NoSQL Datastore for Stream Analytics	2020	[23]
Towards Practical Self-Healing Distributed Databases	2020	[24]
A NoSQL Approach for Aspect Mining of Cultural Heritage Streaming Data	2019	[25]
Near Real-Time Big Data Stream Processing Platform Using Cassandra	2018	[26]
Fault tolerant streaming of live news using multi-node Cassandra	2017	[10]
MySQL to Cassandra conversion engine	2017	[12]
Data modelling for discrete time series data using Cassandra and MongoDB	2016	[27]
Comparison of Relational Database with Document-Oriented Database (MongoDB) for Big Data Applications	2015	[6]

B. Data Analysis

The first step of the analysis is to review each article's Discussion or Conclusion section (if available) and summarize industrial applications mentioned for each data store. Second, all the industry use cases and their performance are recorded and categorized using open coding to create an exhaustive list of industry applications for each data model. Following that, all the opportunities and challenges will be linked to industry use cases and compared between MongoDB and Cassandra.

IV. Results

A. Industry Uses

As data models for handling big data, Cassandra and MongoDB have broad use cases, including geographical data, traffic analysis, Internet of Things (IoT) applications, financial tickers, Web 2.0 and Web 3.0 platforms, e-commerce systems, social networks, and online communities [11]. The reviewed articles explored the following use cases: live news and social media, sensor data, time series data for weather forecasting, and incorporating SQL queries in MongoDB and Cassandra database management systems.

1) Streaming Data: Live News and Social Media. According to Vonitsano, et al. [25], "[t]he colossal measure of information made by a huge number of sensors and the shipment of those information records at the same time are characterized as streaming data." Vonitsano et al. experimented with aspect mining of cultural heritage content on Twitter using Spark streaming architecture as well as Cassandra. In this experiment, Cassandra facilitates cultural content management that analyzes opinions and reviews on social media. With scalability and schemaless data storage, Cassandra manages, processes, and evaluates high volume of hybrid data in real-time. After the storing procedure takes place, the system mainly performs an online aspect mining procedure that applies LDA model to accurately extract and categorize aspects from real-time data on social media.

In addition, Dhingra et al.'s [10] research shows how Cassandra can be used as the underlying data store for a news application, "SmartNews," designed to stream live news data. Their experiment demonstrates Cassandra's ability to manipulate large data from 70+ news sources across 4 nodes. The experiment also tested Cassandra's property of "No Single Point of Failure" by intentionally failing two nodes, and the data can still be retrieved successfully from other nodes.

Furthermore, Lynch and Joshi [24] explained a concrete architecture for building self healing distributed databases using Cassandra. Cassandra is not built with self-healing properties, and deployment at large scale can be complex and hard to manage without support for self-healing. Lynch and Joshi 's research [24] shows how a small number of rules-based agents can be composed together to form a self-healing system for Cassandra.

Based on the urgency of big data transferring, including social media feeds, stock market, and sensors, data need to be analyzed before they are stored in its entirety. Cassandra is optimized for near real-time data storage and processing to analyze streams of data by ingesting stream to database and running ad hoc low latency query [26]. This review shows that Cassandra has advantages in supporting streaming data for content management.

2) Time Series: Weather and Traffic. According to Ramesh et al. [27], Cassandra is the preferred data model for collecting and analyzing large volumes of discrete time series data, as data are inserted into the database sequentially. In their comparison between MongoDB and Cassandra for time series data, every weather work station was characterized by a unique identifier, and they recorded the wind speed to be stored at regular intervals in Cassandra and MongoDB. In this experiment, Cassandra proves its performance in fast retrieval of data and easy scalability.

Based on its rich query language and variable schema, MongoDB is also a good choice for storing the time series data [27]. Hamami et al.'s research [22] aims to build a system that counts vehicles from open CCTV for traffic online in Bali Tower Public Streaming. In addition to use historical data in MongoDB to analyze traffic patterns, data can be visualized in real time to facilitate local authority to understand the vehicle traffic pattern. Data extracted from CCTV streams are highly unstructured data to be process by CNN algorithm. MongoDB is chosen to handle the unstructured big data, because it distributes data into multiple machines horizontally and stores multiple forms of data format.

3) Sensor Data: Spectrum Monitoring and Equipment Fault Detection. A significant source of big data is IIoT(Industrial Internet of Things) sensor systems installed in manufacturing systems, power plants and transportation systems [23]. According to Mahmood and Risch [20], sensor streams are produced by large numbers of diverse types of sensors that are geographically distributed, and these sensor data are highly heterogeneous. These sensor streams are the foundation of the concept of IIoT, and they pose substantial challenges for the existing DBMS technology to provide scalable analytics.

Mahmood and Risch [20] proposed a system to predict online maintenance of industrial equipment using the sensors installed on the equipment that detect abnormal sounds from the equipment. Using forklifts deployed at multiple worksites as an example, this proposed system can avoid mechanical malfunctioning of forklifts' brake systems using a real-time analytic model that analyzes audio streams from the sensors. As a NoSQL DBMS, MongoDB is highly distributed and is used as the database for this system for both online and offline analytics. In this use case, MongoDB's limitation in analytics support is complemented by the extended query processing (EQP) system that allows high-volume stream injection of anomaly streams into MongoDB.

As a highly contended source, radio spectrum monitoring data are stored in both MongoDB and Cassandra to test their performance in the experiment by Baruffa et al. [21]. Their proposed system offers Spectrum Sensing as a Service (*SSaS* to provide access and management of radio spectrum data. Spectrum sensing creates data that allows identifying specific usage patterns in different dimensions (time, frequency, space, angle) and where

free spectrum portions can be found [21]. It cannot be handled by a relational DBMS that does not support full-clustering features, and their experiments show that Cassandra outperforms MongoDB in general when processing sensor data.

4) Incorporate SQL to NoSQL. In reality, many applications still can't avoid using SQL queries. Fu and Widagdo's research [19] shows how to develop a system supported by Cassandra that performs spatial data retrieval from Cassandra using SQL queries by transforming an input SQL query into one or more CQL queries. This is very helpful because 80% of all stored data in corporate databases are spatial data [2].

On the same note, Gopalan et al. [12] developed an system to convert relational DBMS based on MySQL to NoSQL DBMS Cassandra. They designed an algorithm to migrate data as well as schemas from MySQL to Cassandra by creating a dummy column in Cassandra that contains the Primary Key in MySQL. This system provides a tool to switch SQL to NoSQL for better scalability and performance.

The same transferring is happening by utilizing MongoDB. Mahmood et al. [23] proposed a system that enables query-based online data stream analytics over persisted data streams stored/logged in distributed NoSQL datastores. They experimented with a main-memory DBMS and use it to provide online query that process data analytics engine in front of the MongoDB distributed NoSQL datastore to support large scale distributed data analytics over persisted data streams.

B. Visualization of Findings

Findings from the systematic review is briefly visualized in Figure 1. A full-sized figure is included at the end of this article.

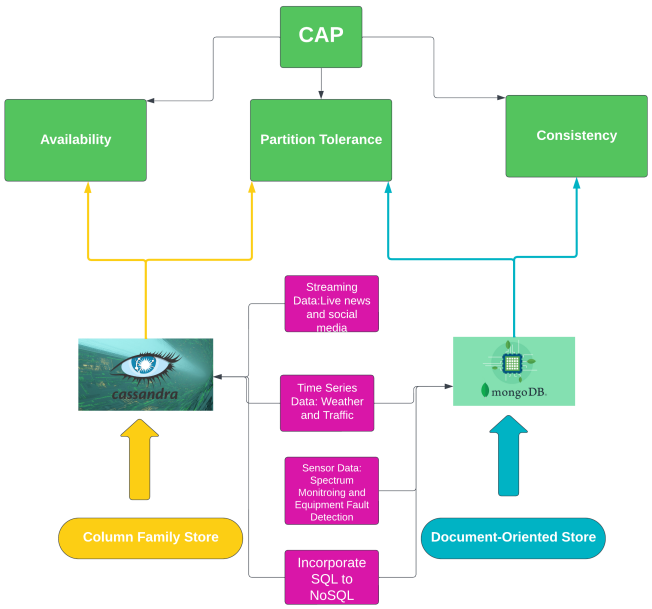

Figure 1: Cassandra and MongoDB Comparison

V. Discussion

Given that there are very few characterized norms for NoSQL databases, no two NoSQL databases are equivalent [8]. The difference between MongoDB and Cassandra can be obvious in their performance with different data types. For example, Anusha et al. [15] noted that "Cassandra performs better with heavy data loads as it can support multiple maser nodes in a cluster whereas MongoDB is best for workloads with huge unstructured data." In Yedilkhan's experiment [13], MongoDB proved to be faster than Cassandra with large data insertion. But when deletion is executed, the performance of MongoDB drops sharply with the growth of the data, and Cassandra behaves best. The same is with search execution. Yedilkhan's study [13] shows Cassandra's performance in search is stronger than MongoDB as time to search for a single record in MongoDB increases rapidly as data grow.

The decision on whether to use MongoDB or Cassandra varies based on the need of the system or application being developed. For example, both MongoDB and Cassandra are great platforms with good scalability, which is a requirement for storing large time series data. Cassandra is advantageous in fast retrieval of data using row key, and MongoDB excels in its rich query language and flexible schema design [27].

A. Replication and Fault Tolerance

Regarding replication, Anusha et al. [15] noted that MongoDB practices master-slave whereas Cassandra has peer-to-peer replication (multi-master). Vonitsanos et al. [25] demonstrated that Cassandra performs advanced custom replication that saves duplicates of the data on all nodes participating in a Cassandra ring. As a result, when one node is shut down, at least one copy of the data that the node has will be accessible to another cluster node. In summary, Cassandra performs better with heavy data loads but MongoDB is the best choice for workloads with huge unstructured data [15, 18].

B. The Theorem of CAP

According to Stonebraker [5], a distributed system can have only two out of the three components of CAP: consistency, availability, and partition tolerance, and there are inevitable theoretical limits on what is possible in the high availability arena. As explained by Yedilkhan et al. [13], "NoSQL systems prioritize scalability and fault tolerance, often at the expense of reduced data consistency and transactional semantics" (p. 483).

Anusha et al's study [15] shows that MongoDB in general achieves consistency and partition tolerance (CP), whereas Cassandra follows partial tolerance and availability (PA) (1). According to Sharma et al. [2], Cassandra does not support join, but it does provides high data availability by accepting denormalization of data. To support availability, Cassandra duplicates data on multiple nodes to help ensure reliability and fault tolerance [2]. These findings provide key metrics as to when to use what data store.

C. ACID and NoSQL

ACID (Atomicity, Consistency, Isolation, and Durability) are SQL DBMS properties. According to Capris et al. [18], NoSQL databases were born to meet performance needs, leaving other details like atomicity in the background. Strict data schemas and integrity constraints are missing from NoSQL systems, as NoSQL being more straightforward than relational databases [13].

However, with the efforts of incorporating SQL into NoSQL, some principles of ACID can now be accomplished by NoSQL databases. According to Ramesh et al. [27], compared with relational databases, Cassandra also supports ACID properties and has very fast write speed, and MongoDB has a rich query language that is no less powerful than relational databases. The cases reviewed in this research show that SQL is now being integrated into NoSQL using different techniques such as wrapping and converting SQL to NoSQL, or transforming an input SQL query into CQL queries in Cassandra.

VI. Conclusion and Future Studies

The magnitude of data being collected on the Internet has been both exciting and concerning. As noted by former Google CEO Eric Schmidt in 2010, "Every two days now we create as much information as we did from the dawn of civilization up until 2003," which is about five exabytes of data [28]. In 2014, Sharma et al. [2] stated that "the rampant usage of social media such as Twitter, Facebook, etc. is the sole contributor of approximately 90% of the total data available today" (p.138). As the price for data collection and storage decreased and the potential profitability of data grows, NoSQL tools have become the ideal choice for real-time data processing to handle big data.

This systematic review studies the most recent industry uses of two NoSQL stores, MongoDB and Cassandra. The results reveals their industry uses in three major areas: streaming data, time series data, and sensor data. The industry uses reflect special advantages in NoSQL data stores such as fault tolerance and scalability, and their possibilities to incorporate ACID, which are typical principles for relational databases. Cassandra outperforms MongoDB as the size of data increases, while MongoDB outperforms Cassandra in handling complicated data structure [15,21,29]. MongoDB and Cassandra are both efficient for Content Management Systems [15, 26].

Looking into the future, the combination of relational DBMS and NoSQL will be more popular. Mostajabi et al. [11] introduces a hierarchical data model called NewSQL that combines goals of both relational DBMS and NoSQL DBMS by providing horizontal scalability similar to NoSQL while maintaining ACID properties. At the same time, cross-data store transactions using many-database systems are on the rise [30]. These changes will provide more choices for applications seeking powerful and safe data storage and management.

References

[1] D. Laney, "3d data management: Controlling data volume, velocity and variety," *META Group Research Note*, no. 6, 2001.

[2] S. Sharma, U. S. Tim, J. Wong, S. Gadia, and S. Sharma, "A brief review on leading big data models," *Data Science Journal*, vol. 13, no. 4, pp. 138–157, 2014.

[3] R. Maheshwari, "3 v's or 7 v's - what's the value of big data?," tech. rep., 2015.

[4] X. Wu, Z. Du, S. Dai, and Y. Liu, "The fault tolerance of big data systems," *Communications in Computer and Information Science*, vol. 686, p. 65–74, 2017.

[5] M. Stronbraker and R. Cattell, "10 rules for scalable performance in 'simple operation' datastores," *Communications of the ACM*, vol. 54, no. 6, pp. 72–80, 2011.

[6] S. Chickerur, A. Goudar, and A. Kinnerkar, "Comparison of relational database with document-oriented database (mongodb) for big data applications," in *2015 8th International Conference on Advanced Software Engineering & Its Applications (ASEA)*, pp. 41–47, 2015.

[7] M. Stonebraker, "Sql databases v. nosql databases," *COMMUNICATIONS OF THE ACM*, vol. 53, pp. 10–11, APR 2010.

[8] S. Palanisamy and P. SuvithaVani, "A survey on rdbms and nosql databases mysql vs mongodb," in *2020 International Conference on Computer Communication and Informatics (ICCCI)*, pp. 1–7, Jan 2020.

[9] M. M. Najafabadi, F. Villanustre, T. M. Khoshgoftaar, N. Seliya, R. Wald, and E. Muharemagic, "Deep learning applications and challenges in big data analytics," *Journal of Big Data*, vol. 2, no. 1, pp. 1–21, 2015.

[10] S. Dhingra, S. Sharma, P. Kaur, and C. Dabas, "Fault tolerant streaming of live news using multi-node cassandra," in *2017 Tenth International Conference on Contemporary Computing (IC3)*, pp. 1–5, 2017.

[11] F. Mostajabi, A. A. Safaei, and A. Sahafi, "A systematic review of data models for the big data problem," *IEEE Access*, vol. 9, pp. 128889–128904, 2021.

[12] M. G. Gopalan, C. Prasanna, Y. V. Srihari Krishna, B. Shanthini, and A. Arulkumar, "Mysql to cassandra conversion engine," in *2017 Third International Conference on Sensing, Signal Processing and Security (ICSSS)*, pp. 503–508, May 2017.

[13] D. Yedilkhan, A. Mukasheva, D. Bissengaliyeva, and Y. Suynullayev, "Performance analysis of scaling nosql vs sql: A comparative study of mongodb, cassandra, and postgresql," in *2023 IEEE International Conference on Smart Information Systems and Technologies (SIST)*, pp. 479–483, May 2023.

[14] J. Carpenter, "How the world caught up with apache cassandr," n.d.

[15] K. Anusha, N. Rajesh, M. Kavitha, and N. Ravinder, "Comparative study of mongodb vs cassandra in big data analytics," in *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, pp. 1831–1835, April 2021.

[16] J. Schultz, "Cassandra use cases: When to use and when not to use cassandra," *Pythian*, 2018, March 21.

[17] H. Jain, "Best 7 real-world mongodb use cases," *Hevo*, August 28, 2023.

[18] T. Capris, P. Melo, N. M. Garcia, I. M. Pires, and E. Zdravevski, "Comparison of sql and nosql databases with different workloads: Mongodb vs mysql evaluation," in *2022 International Conference on Data Analytics for Business and Industry (ICDABI)*, pp. 214–218, Oct 2022.

[19] W. Fu and T. E. Widagdo, "Sql interface development for spatial data retrieval on cassandra," in *2022 International Conference on Data and Software Engineering (ICoDSE)*, pp. 138–143, 2022.

[20] K. Mahmood and T. Risch, "Scalable real-time analytics for iot applications," in *2021 IEEE International Conference on Smart Computing (SMARTCOMP)*, pp. 404–406, 2021.

[21] G. Baruffa, M. Femminella, M. Pergolesi, and G. Reali, "Comparison of mongodb and cassandra databases for spectrum monitoring as-a-service," *IEEE Transactions on Network and Service Management*, vol. 17, no. 1, pp. 346–360, 2020.

[22] F. Hamami, I. A. Dahlan, S. W. Prakosa, and K. F. Somantri, "Big data analytics for processing real-time unstructured data from cctv in traffic management," in *2020 International Conference on Data Science and Its Applications (ICoDSA)*, pp. 1–5, 2020.

[23] K. Mahmood, K. Orsborn, and T. Risch, "Wrapping a nosql datastore for stream analytics," in *2020 IEEE 21st International Conference on Information Reuse and Integration for Data Science (IRI)*, pp. 301–305, 2020.

[24] J. Lynch and D. Ashok Joshi, "Towards practical self-healing distributed databases," in *2020 IEEE Infrastructure Conference*, pp. 1–4, 2020.

[25] G. Vonitsanos, A. Kanavos, A. Mohasseb, and D. Tsolis, "A nosql approach for aspect mining of cultural heritage streaming data," in *2019 10th International Conference on Information, Intelligence, Systems and Applications (IISA)*, pp. 1–4, 2019.

[26] G. Pal, G. Li, and K. Atkinson, "Near real-time big data stream processing platform using cassandra," in *2018 4th International Conference for Convergence in Technology (I2CT)*, pp. 1–7, 2018.

[27] D. Ramesh, A. Sinha, and S. Singh, "Data modelling for discrete time series data using cassandra and mongodb," in *2016 3rd International Conference on Recent Advances in Information Technology (RAIT)*, pp. 598–601, 2016.

[28] M. Siegler, "Eric schmidt: Every 2 days we create as much information as we did up to 2003," August 4, 2010.

[29] V. Abramova and J. Bernardino, "Nosql databases: Mongodb vs cassandra," in *Proceedings of the International C* Conference on Computer Science and Software Engineering*, C3S2E '13, (New York, NY, USA), p. 14–22, Association for Computing Machinery, 2013.

[30] P. Kraft, Q. Li, X. Zhou, P. Bailis, M. Stonebraker, M. Zaharia, and X. Yu, "Epoxy: Acid transactions across diverse data stores," *Proc. VLDB Endow.*, vol. 16, p. 2742–2754, jul 2023.